Test Implementation: Contact Manager System

Software Testing, Validation and Verification (CSE265)

Project: C++ Console-based Contact Manager System

Under the Supervision of: Dr. Gehad Mohey

Version: 1.0

Date: May 10, 2026

Student Name:	ID:
Mazen Yasser	247861
Jasmine Ali	240247
Sandy Atef	245077
Mariam Mohamed Reyad	245171
Marwan Ehab	248889

Table of Contents

1.Introduction

The testing process was implemented using **Google Test** and **Google Mock** in Visual Studio. A separate testing project was created to test the main functionalities of the Contact Management System without affecting the original application files or the real contacts.json data file.

The implementation focused on testing the main modules of the system, especially the ContactService, ContactRepository, and the interaction between them. The tests were divided into **unit testing** and **integration testing**.

2.Unit Testing

Unit testing was applied mainly to the ContactService module. The purpose of these tests was to verify that each function works correctly in isolation, such as adding, searching, updating, deleting contacts, validating inputs, and generating unique IDs.

A **test fixture** was used to avoid repeating the setup code in every test case. The fixture creates a test repository and service object before each test. It also uses a separate JSON file called unit_test_contacts.json so that the real contact data is not affected.

The SetUp() function runs before each test case. It removes any old test file and initializes the service. This ensures that every test starts with clean data. The TearDown() function runs after each test case and deletes the test file to prevent test data from affecting other tests.

```
#include "pch.h"

#include "gtest/gtest.h"
#include "gmock/gmock.h"

#include "Contact.h"
#include "ContactRepository.h"
#include "ContactService.h"

#include <cstdio>
#include <string>
#include <vector>

using ::testing::SizeIs;

// Unit testing test fixture

class ContactServiceUnitTest : public ::testing::Test
{
protected:
    const std::string testFile = "unit_test_contacts.json";

    ContactRepository repo;
    ContactService service;

    ContactServiceUnitTest()
        : repo(testFile), service(repo)
    {}

    void SetUp() override
    {
        std::remove(testFile.c_str());
        service.initialize();
    }

    void TearDown() override
    {
        std::remove(testFile.c_str());
```

```
    }
};

////////////////////////////////////////////////////////////////////////////////////////////////////
// UNIT TESTING

// Add Testing

// UT-01: Add valid contact
TEST_F(ContactServiceUnitTest, UT01_AddValidContact)
{
    bool result = service.addContact("Ali", "01012345678");

    EXPECT_TRUE(result);

    std::vector<Contact> contacts = service.viewContacts();

    ASSERT_THAT(contacts, SizeIs(1));
    EXPECT_EQ(contacts[0].getName(), "Ali");
    EXPECT_EQ(contacts[0].getPhone(), "01012345678");
}

// UT-02: Add contact with empty name
TEST_F(ContactServiceUnitTest, UT02_AddEmptyName)
{
    bool result = service.addContact("", "01012345678");

    EXPECT_FALSE(result);
    EXPECT_THAT(service.viewContacts(), SizeIs(0));
}

// UT-03: Add contact with empty phone
TEST_F(ContactServiceUnitTest, UT03_AddEmptyPhone)
{
    bool result = service.addContact("Ali", "");

    EXPECT_FALSE(result);
    EXPECT_THAT(service.viewContacts(), SizeIs(0));
}

// UT-04: Generate unique ID
TEST_F(ContactServiceUnitTest, UT04_UniqueID)
{
    service.addContact("Ali", "01012345678");
    service.addContact("Mona", "01112345678");
    service.addContact("Sara", "01212345678");

    std::vector<Contact> contacts = service.viewContacts();

    ASSERT_THAT(contacts, SizeIs(3));

    EXPECT_NE(contacts[0].getId(), contacts[1].getId());
    EXPECT_NE(contacts[0].getId(), contacts[2].getId());
    EXPECT_NE(contacts[1].getId(), contacts[2].getId());
}

// UT-05: Validate phone format
TEST_F(ContactServiceUnitTest, UT05_InvalidPhone)
{
    bool result = service.addContact("Ali", "abc123");

      EXPECT_FALSE(result);
      EXPECT_THAT(service.viewContacts(), SizeIs(0));
 }

 // UT-06: Add duplicate contact
 TEST_F(ContactServiceUnitTest, UT06_AddDuplicateContact)
 {
      bool firstAdd = service.addContact("Ali", "01012345678");
      bool duplicateAdd = service.addContact("Ali", "01012345678");

      EXPECT_TRUE(firstAdd);
```

```
111.     EXPECT_FALSE(duplicateAdd);
112. 
113.     std::vector<Contact> contacts = service.viewContacts();
114. 
115.     ASSERT_THAT(contacts, SizeIs(1));
116.     EXPECT_EQ(contacts[0].getName(), "Ali");
117.     EXPECT_EQ(contacts[0].getPhone(), "01012345678");
118. }
119. 
120. // Search Testing
121. 
122. // UT-07: Search existing contact
123. TEST_F(ContactServiceUnitTest, UT07_SearchExistingContact)
124. {
125.     service.addContact("Ali", "01012345678");
126.     service.addContact("Mona", "01112345678");
127. 
128.     std::vector<Contact> result = service.search("Ali");
129. 
130.     ASSERT_THAT(result, SizeIs(1));
131.     EXPECT_EQ(result[0].getName(), "Ali");
132. }
133. 
134. 
135. 
136. // UT-08: Search partial name
137. TEST_F(ContactServiceUnitTest, UT08_SearchPartialName)
138. {
139.     service.addContact("Ali", "01012345678");
140.     service.addContact("Alaa", "01112345678");
141.     service.addContact("Mona", "01212345678");
142. 
143.     std::vector<Contact> result = service.search("Al");
144. 
145.     ASSERT_THAT(result, SizeIs(2));
146.     EXPECT_EQ(result[0].getName(), "Ali");
147.     EXPECT_EQ(result[1].getName(), "Alaa");
148. }
149. 
150. // UT-09: Search non-existing contact
151. TEST_F(ContactServiceUnitTest, UT09_SearchNonExistingContact)
152. {
153.     service.addContact("Ali", "01012345678");
154. 
155.     std::vector<Contact> result = service.search("Omar");
156. 
157.     EXPECT_THAT(result, SizeIs(0));
158. }
159. 
160. // Delete Testing
161. 
162. // UT-010: Delete valid ID
163. TEST_F(ContactServiceUnitTest, UT10_DeleteValidID)
164. {
165.     service.addContact("Ali", "01012345678");
166. 
167.     std::vector<Contact> contacts = service.viewContacts();
168.     ASSERT_THAT(contacts, SizeIs(1));
169. 
170.     int id = contacts[0].getId();
171. 
172.     bool deleted = service.deleteContact(id);
173. 
174.     EXPECT_TRUE(deleted);
175.     EXPECT_THAT(service.viewContacts(), SizeIs(0));
176. }
177. 
178. // UT-11: Delete invalid ID
179. TEST_F(ContactServiceUnitTest, UT11_DeleteInvalidID)
180. {
181.     service.addContact("Ali", "01012345678");
182. 
183.     bool deleted = service.deleteContact(999);
```

```
184.
185.     EXPECT_FALSE(deleted);
186.     EXPECT_THAT(service.viewContacts(), SizeIs(1));
187. }
188.
189. // UT-12: Update valid contact
190. TEST_F(ContactServiceUnitTest, UT12_UpdateValidContact)
191. {
192.     service.addContact("Ali", "01012345678");
193.
194.     std::vector<Contact> contacts = service.viewContacts();
195.     ASSERT_THAT(contacts, SizeIs(1));
196.
197.     int id = contacts[0].getId();
198.
199.     bool updated = service.updateContact(id, "Ahmed", "01111111111");
200.
201.     EXPECT_TRUE(updated);
202.
203.     std::vector<Contact> updatedContacts = service.viewContacts();
204.
205.     ASSERT_THAT(updatedContacts, SizeIs(1));
206.     EXPECT_EQ(updatedContacts[0].getName(), "Ahmed");
207.     EXPECT_EQ(updatedContacts[0].getPhone(), "01111111111");
208. }
209.
210. // UT-13: Update invalid ID
211. TEST_F(ContactServiceUnitTest, UT13_UpdateInvalidID)
212. {
213.     service.addContact("Ali", "01012345678");
214.
215.     bool updated = service.updateContact(999, "Ahmed", "01111111111");
216.
217.     EXPECT_FALSE(updated);
218.
219.     std::vector<Contact> contacts = service.viewContacts();
220.
221.     ASSERT_THAT(contacts, SizeIs(1));
222.     EXPECT_EQ(contacts[0].getName(), "Ali");
223.     EXPECT_EQ(contacts[0].getPhone(), "01012345678");
224. }
```

3.Integration Testing

Integration testing was used to test the interaction between the ContactService and ContactRepository modules. These tests verify that operations performed through the service layer are correctly saved to and loaded from the JSON file.

A separate fixture was created for integration tests. It uses a different test file called integration_test_contacts.json.

```
    ////////////////////////////////////////////////////////////////////////////////////////////
    ////////////////////////////////////////////////
225.
226. // Integration Testing test fixture
227.
228. class ContactIntegrationTest : public ::testing::Test
229. {
230. protected:
231.     std::string testFile;
232.     ContactRepository repo;
233.     ContactService service;
234.
235.     ContactIntegrationTest()
236.         : testFile("integration_test_contacts.json"),
237.         repo(testFile),
238.         service(repo)
239.     {}
240.
241.     void SetUp() override
242.     {
243.         std::remove(testFile.c_str());
244.         service.initialize();
245.     }
246.
247.     void TearDown() override
248.     {
249.         std::remove(testFile.c_str());
250.     }
251. };
252.
253. // Integration Testing
254. // IT-01: Add contact integration
255. TEST_F(ContactIntegrationTest, IT01_AddContactIntegration)
256. {
257.     bool result = service.addContact("Ali", "01012345678");
258.
259.     EXPECT_TRUE(result);
260.
261.     ContactRepository newRepo(testFile);
262.     newRepo.load();
263.
264.     std::vector<Contact> contacts = newRepo.getAll();
265.
266.     ASSERT_THAT(contacts, SizeIs(1));
267.     EXPECT_EQ(contacts[0].getName(), "Ali");
268.     EXPECT_EQ(contacts[0].getPhone(), "01012345678");
269. }
270.
271. // IT-02: View contacts integration
272. TEST_F(ContactIntegrationTest, IT02_ViewContactsIntegration)
273. {
274.     service.addContact("Ali", "01012345678");
275.     service.addContact("Mona", "01112345678");
276.
277.     ContactRepository newRepo(testFile);
278.     ContactService newService(newRepo);
279.
280.     newService.initialize();
281.
282.     std::vector<Contact> contacts = newService.viewContacts();
283.     ASSERT_THAT(contacts, SizeIs(2));
```

```
    EXPECT_EQ(contacts[0].getName(), "Ali");
    EXPECT_EQ(contacts[0].getPhone(), "01012345678");
    EXPECT_EQ(contacts[1].getName(), "Mona");
    EXPECT_EQ(contacts[1].getPhone(), "01112345678");
}

// IT-03: Search integration
TEST_F(ContactIntegrationTest, IT03_SearchIntegration)
{
    service.addContact("Ali", "01012345678");
    service.addContact("Mona", "01112345678");

    ContactRepository newRepo(testFile);
    ContactService newService(newRepo);

    newService.initialize();

    std::vector<Contact> result = newService.search("Ali");

    ASSERT_THAT(result, SizeIs(1));
    EXPECT_EQ(result[0].getName(), "Ali");
    EXPECT_EQ(result[0].getPhone(), "01012345678");
}

// IT-04: Update integration
TEST_F(ContactIntegrationTest, IT04_UpdateIntegration)
{
    service.addContact("Ali", "01012345678");

    std::vector<Contact> contacts = service.viewContacts();
    ASSERT_THAT(contacts, SizeIs(1));

    int id = contacts[0].getId();

    bool updated = service.updateContact(id, "Ahmed", "01111111111");

    EXPECT_TRUE(updated);

    ContactRepository newRepo(testFile);
    ContactService newService(newRepo);

    newService.initialize();

    std::vector<Contact> updatedContacts = newService.viewContacts();

    ASSERT_THAT(updatedContacts, SizeIs(1));
    EXPECT_EQ(updatedContacts[0].getName(), "Ahmed");
    EXPECT_EQ(updatedContacts[0].getPhone(), "01111111111");
}

// IT-05: Delete integration ------
TEST_F(ContactIntegrationTest, IT05_DeleteIntegration)
{
    service.addContact("Ali", "01012345678");

    std::vector<Contact> contacts = service.viewContacts();
    ASSERT_THAT(contacts, SizeIs(1));

    int id = contacts[0].getId();

    bool deleted = service.deleteContact(id);

    EXPECT_TRUE(deleted);

    ContactRepository newRepo(testFile);
    ContactService newService(newRepo);

    newService.initialize();

    std::vector<Contact> remainingContacts = newService.viewContacts();

    EXPECT_THAT(remainingContacts, SizeIs(0));
}
```

4.Test Execution

The tests were executed using the Visual Studio Test Explorer. After building the solution successfully, the test cases appeared in Test Explorer and were run automatically. Each test result was shown as either passed or failed.

The test implementation confirms that the main contact operations behave correctly, including adding, viewing, searching, updating, deleting, validation, ID generation, and persistence through JSON file storage.

5.Conclusion

The test implementation successfully validates the main behavior of the Contact Management System. Unit tests were used to test individual service functions, while integration tests were used to verify the interaction between the service layer, repository layer, and JSON file. The use of fixtures helped reduce repeated code and ensured that every test started with clean and isolated data.